
Scheduling

xv6 scheduler

Contents

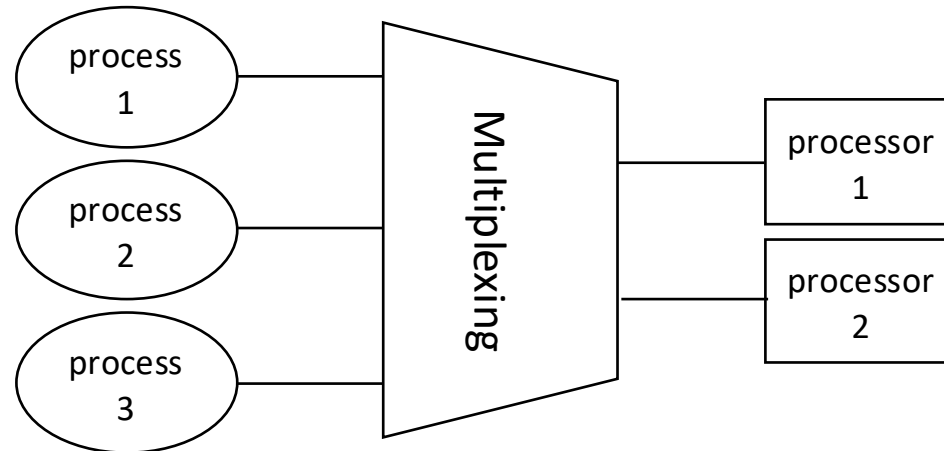
- Context Switch Overview
- Scheduling in xv6
- Practice on Your Own

Context Switch Overview

Why and How Processes Switch

CPU virtualization

- Modern operating systems support running more processes than the number of available processors simultaneously.
- **CPU virtualization** is to provide each process with the illusion that it has its own CPU.
- A common approach to achieve **CPU virtualization** is by multiplexing (time-sharing) the processes onto the hardware processors.

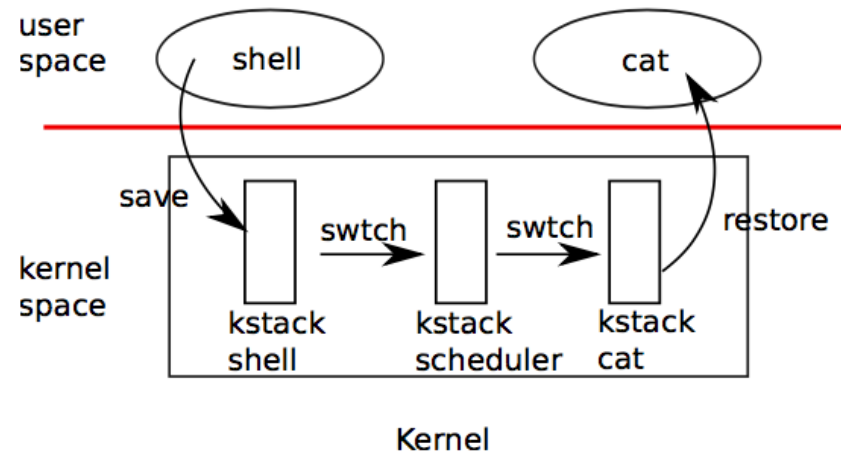


Switching Process

- Multiplexing is achieved by frequently switching each processor from one process to another.
- Switching occurs in two situations:
 - Periodically timer interrupt forces a switch (i.e., the process yields the CPU)
 - Sleep and wakeup mechanism (covered in a later lecture)
- In xv6, timer interrupt handler transparently run **context switch**, which is the standard mechanism of time sharing among multiple processes.
- Locking is necessary to avoid races among the processes involved in context switch.

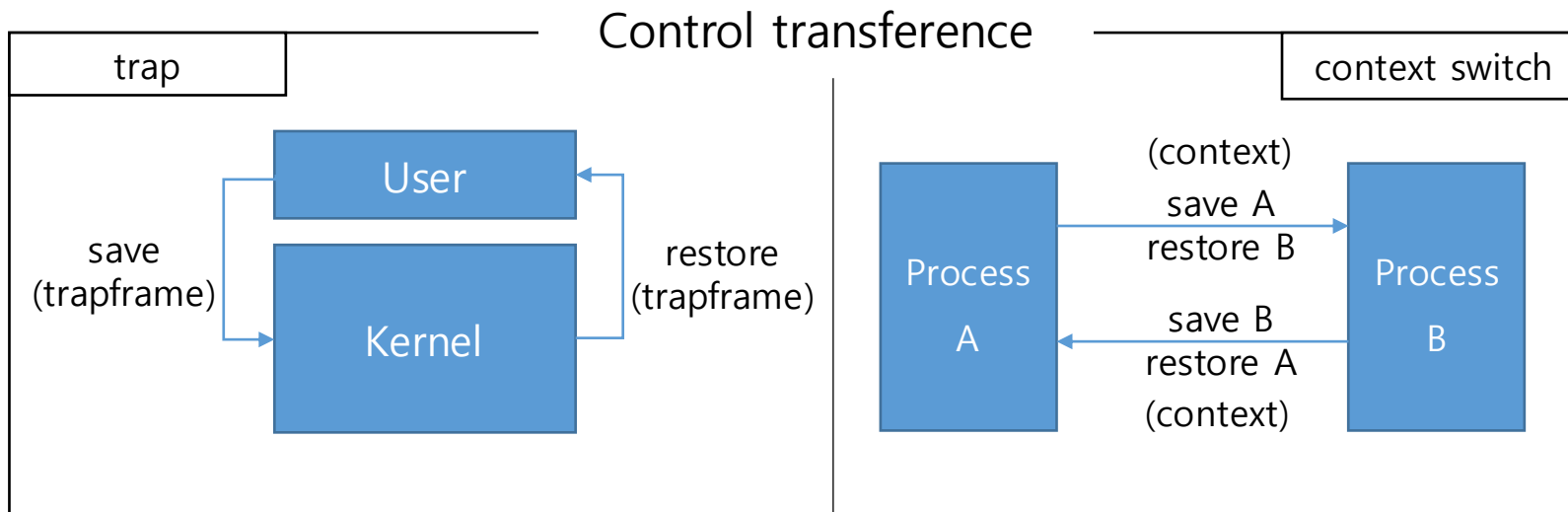
Context Switch

- Every process has its own kernel stack and register set for execution, which is called **context**.
- Context switch means saving old thread's context and restoring the new thread's previously saved context.
- In xv6, switching from one user process to another user process involves two context switches. (switch to CPU's scheduler code in the middle)



struct context

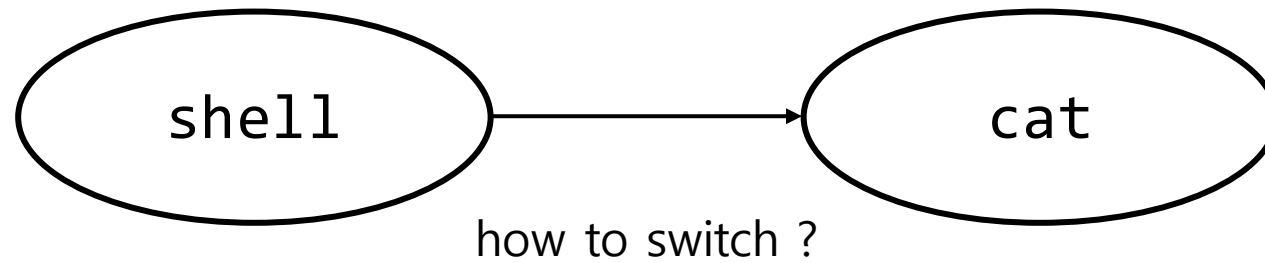
- Saved register for kernel context switches.
 - similar to how struct trapframe is used for trap handling.
- The context switch is also control transference, so registers must be saved and restored.



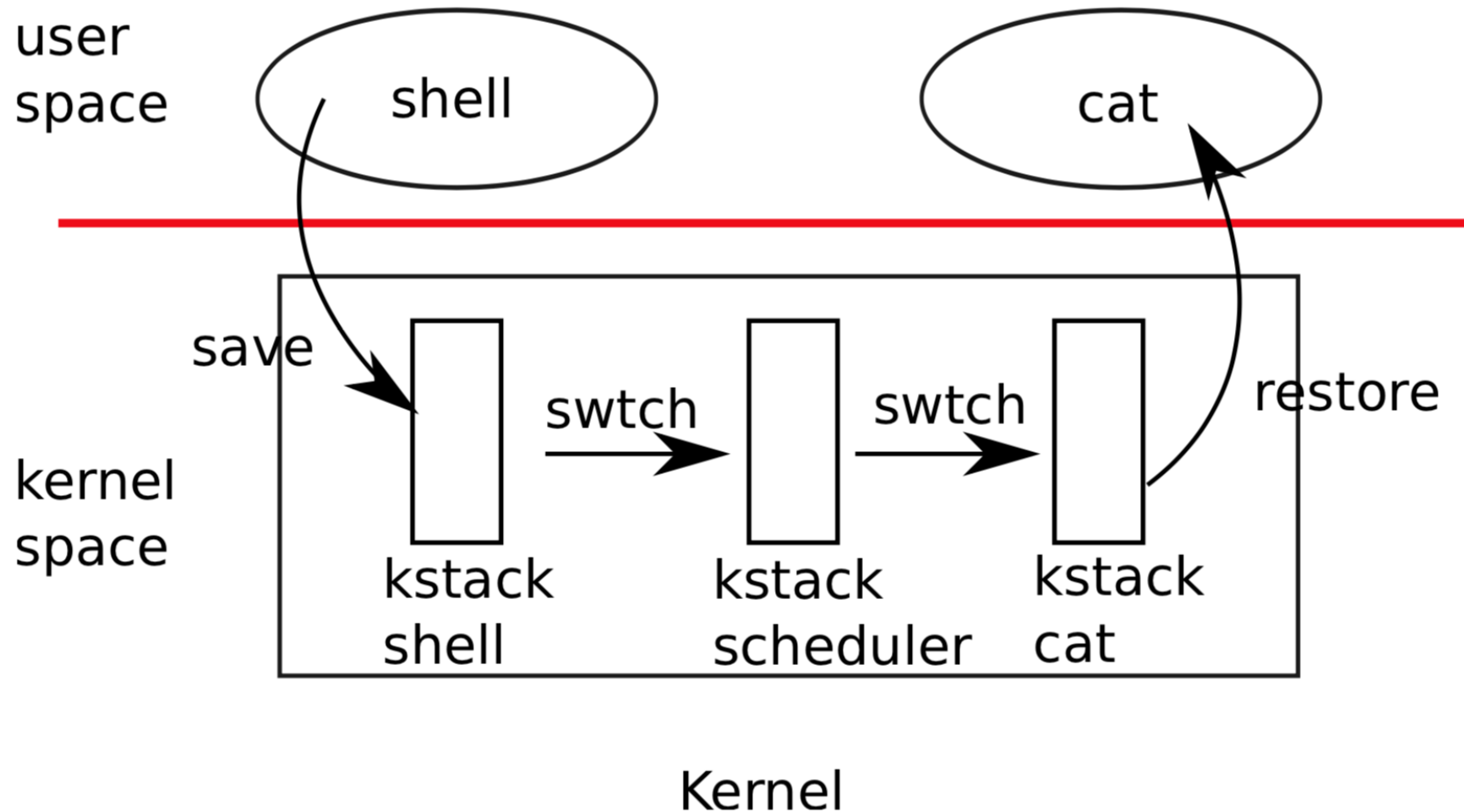
```
$ vim kernel/proc.h

2  struct context {
3      uint64 ra;
4      uint64 sp;
5      ...
6      // callee-saved
7      uint64 s0;
8      uint64 s1;
9      ...
18     uint64 s11;
19 };
```

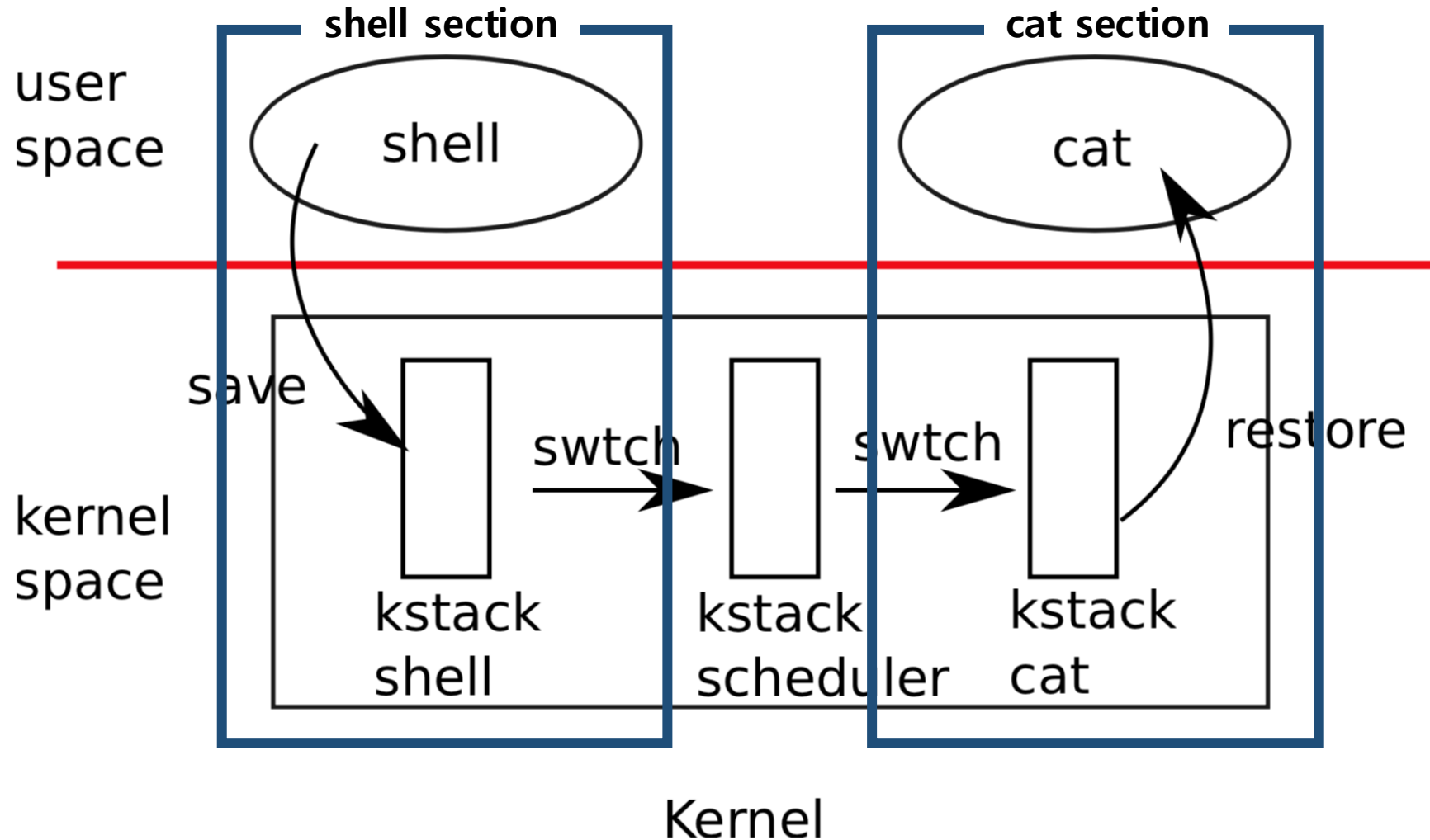
Context Switch in detail



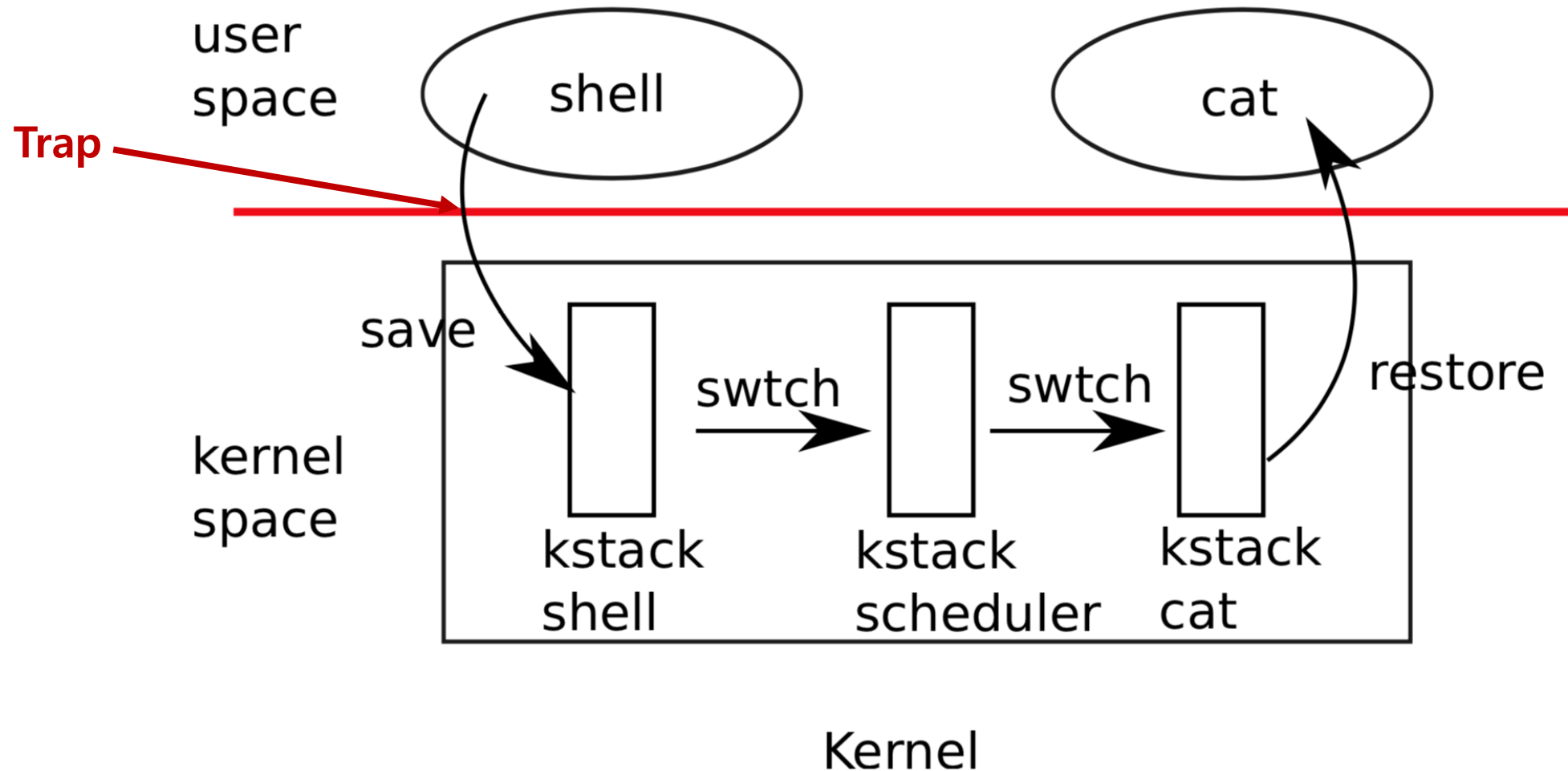
Context Switch in detail



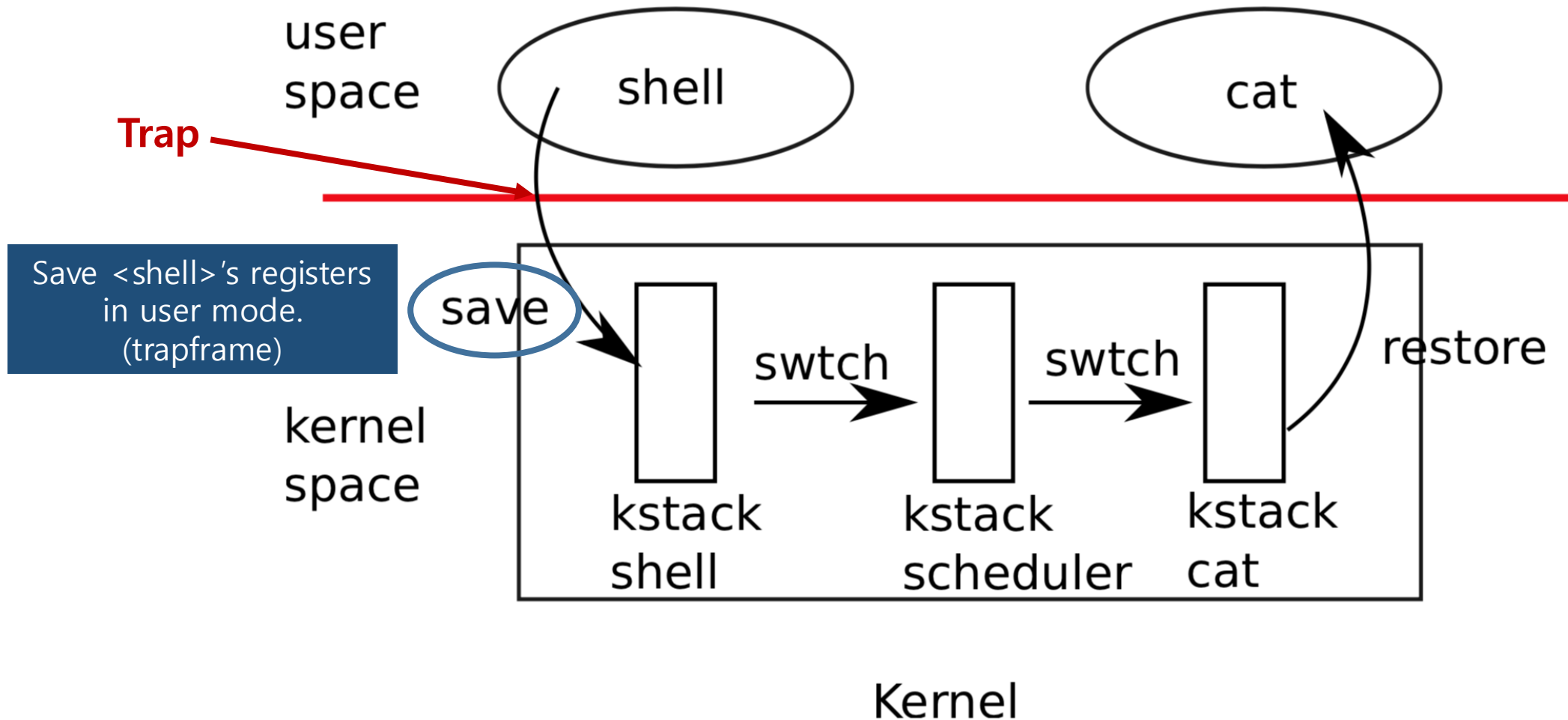
Context Switch in detail



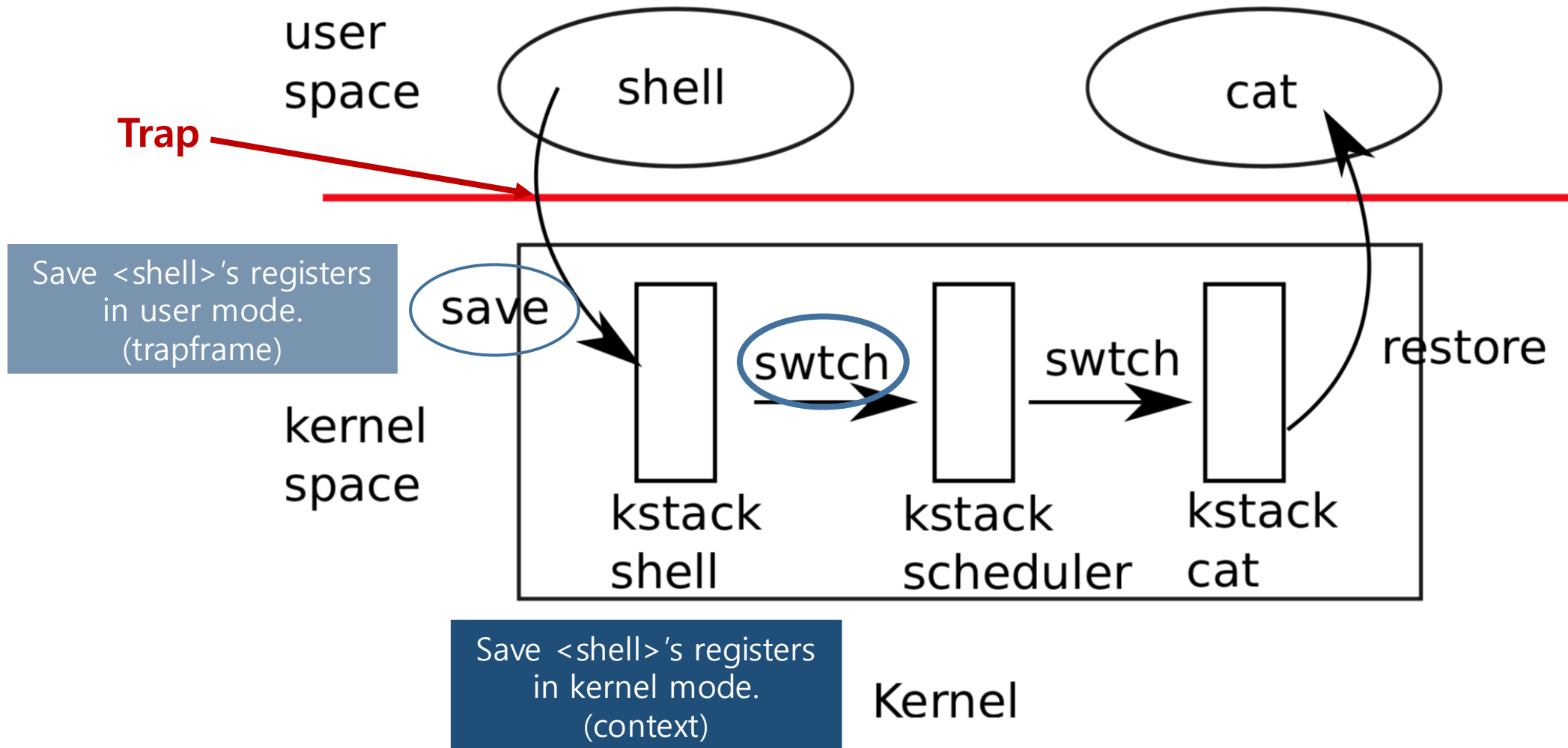
Context Switch in detail



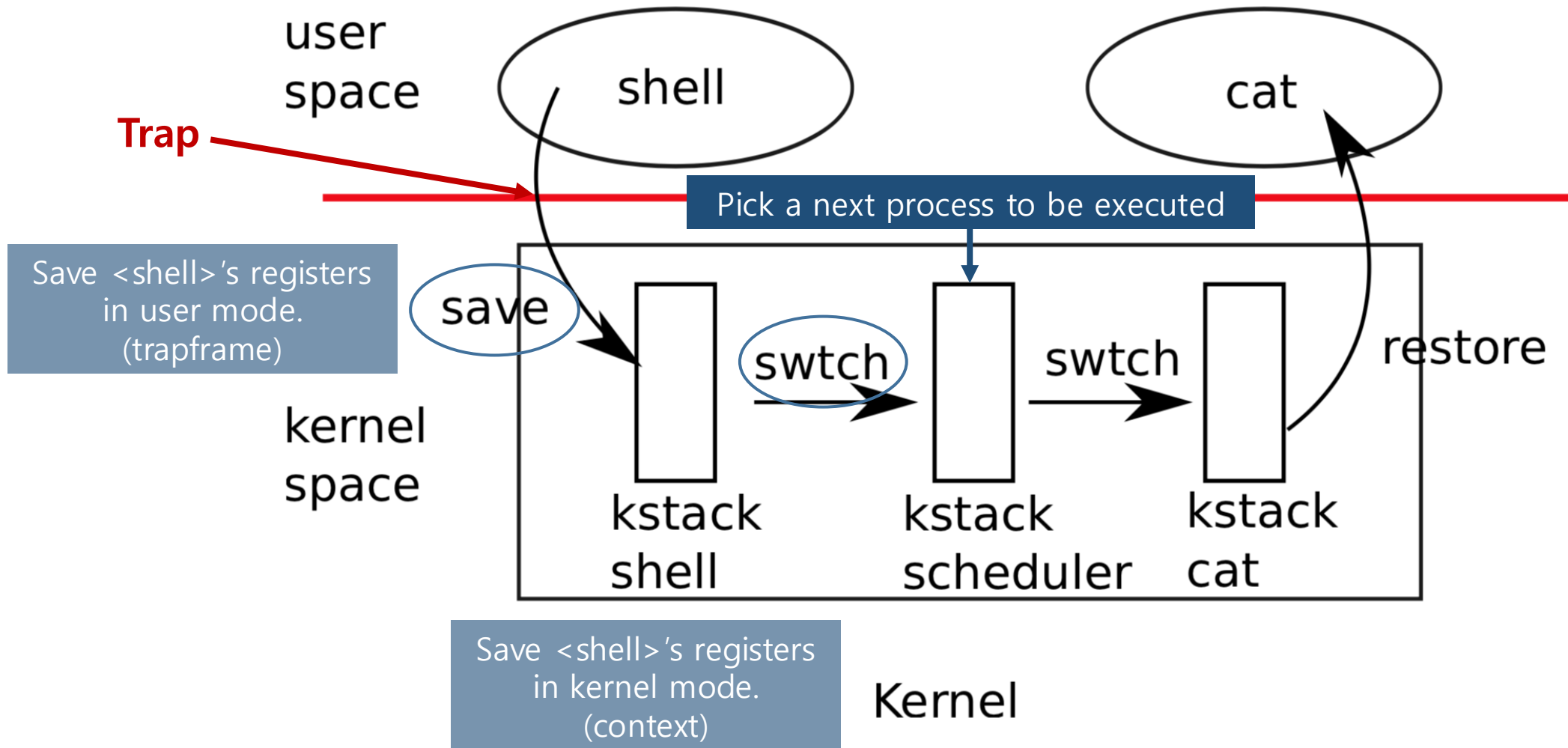
Context Switch in detail



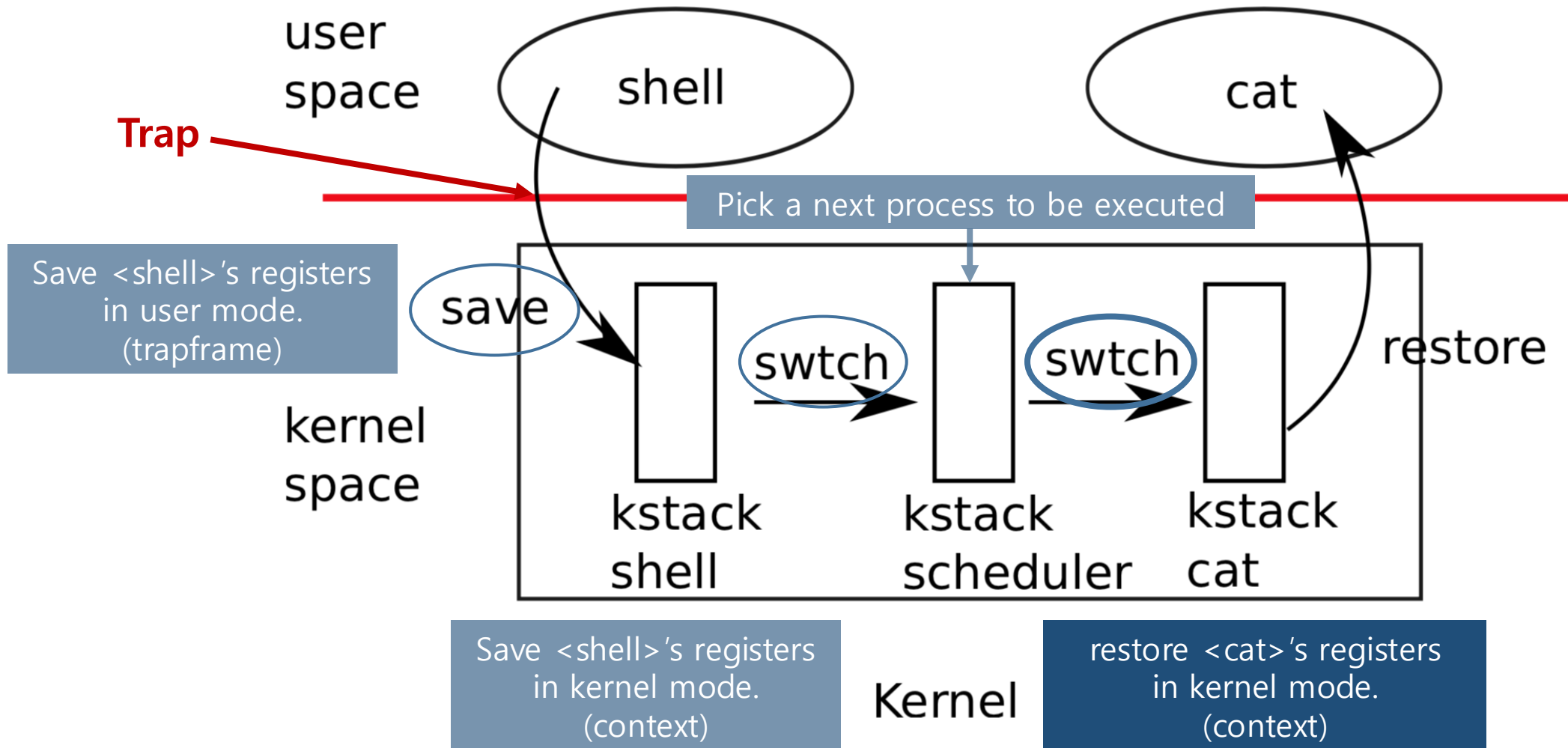
Context Switch in detail



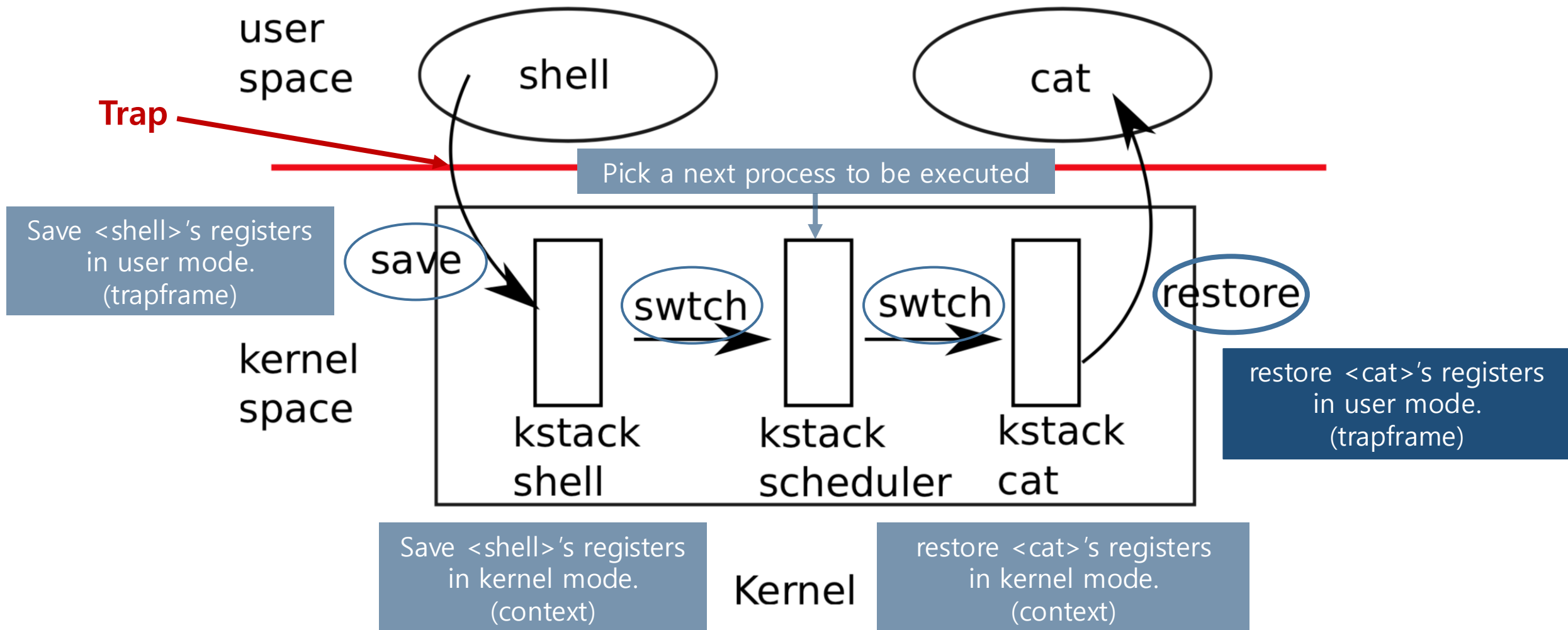
Context Switch in detail



Context Switch in detail



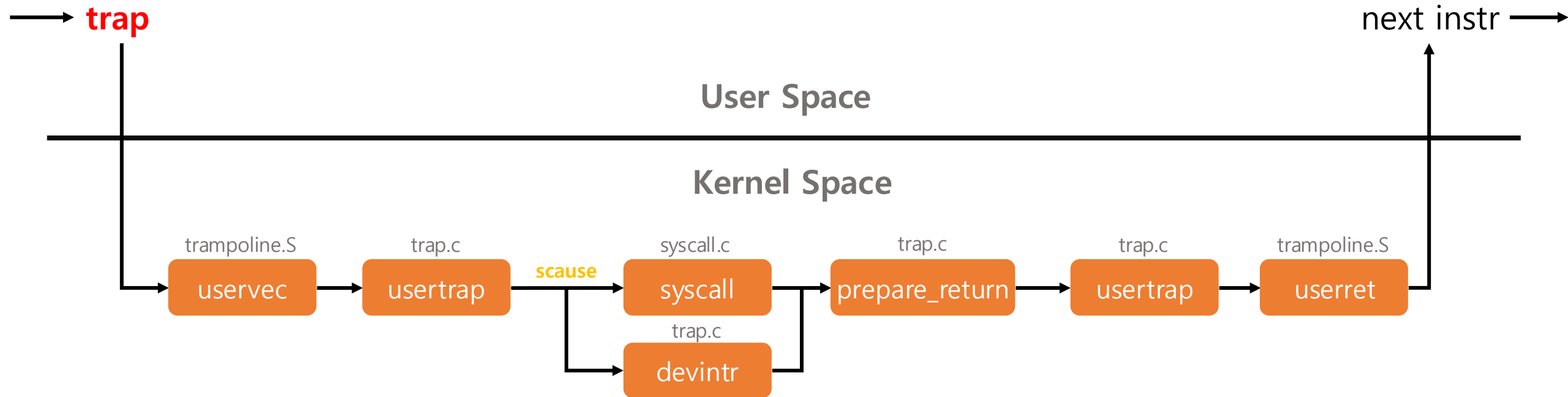
Context Switch in detail



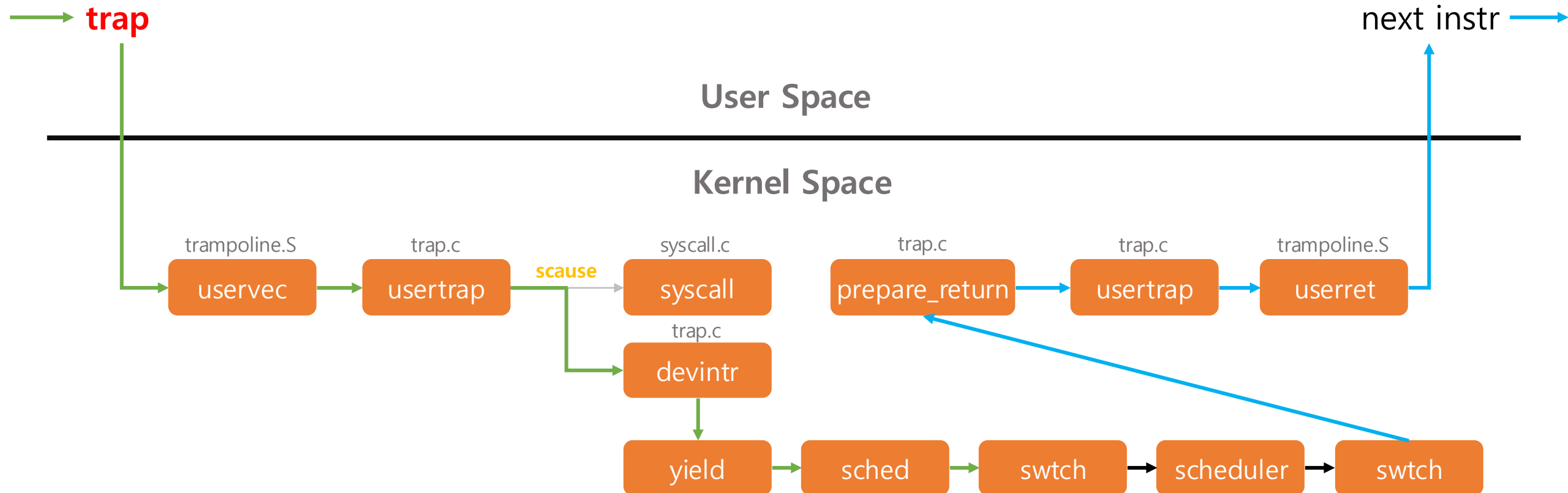
Scheduling in xv6

yield, sched, scheduler, and switch

Recall: Traps from User Space



Overview: Context Switch by Timer Interrupt



Timer Interrupt

- A clock chip generates a timer interrupt every ~100 ms (called a **tick**) in xv6.
- Each timer interrupt increments the global variable **ticks** in `clockintr()`.

```
$ vim kernel/trap.c
```

```
185  int
186  devintr()
187  {
188      uint 64 scause = r_scause()
189
190      if(scause == 0x8000000000000009L){
191          ... /* handle external device interrupt */
211      else if (scause == 0x8000000000000005L){
212          // timer interrupt.
213          clockintr();
214          return 2;
215      } else {
```

```
$ vim kernel/trap.c
```

```
164  void
165  clockintr()
166  {
167      if(cpuid()==0){
168          acquire(&tickslock);
169          ticks++;
170          wakeup(&ticks);
171          release(&tickslock);
172      }
173      ...
176      w_stimecmp(r_time() + 1000000);
177  }
```

yield

- A process that wants to give up CPU calls yield.

```
$ vim kernel/trap.c
```

```
37 void
38 usertrap(void)
39 {
...
54     if(r_scause() == 8){ //system call
69     else if ((which_dev = devintr()) != 0){
...
78     }
...
84     if(which_dev == 2)
85         yield();
...
87     prepare_return();
...
93     return satp;
94 }
```

```
$ vim kernel/trap.c
```

```
135 void
136 kerneltrap(void)
137 {
138     int which_dev = 0;
...
148     if((which_dev = devintr()) == 0){
...
155         if(which_dev == 2 && myproc() != 0)
156             yield();
...
163 }
```

yield

- A process that wants to give up CPU calls yield.
- `yield()` acquires process lock, change the process state, and calls `sched()`.

```
$ vim kernel/proc.c

493 void
494 yield(void)
495 {
496     struct proc *p = myproc();
497     acquire(&p->lock);
498     p->state = RUNNABLE;
499     sched();
500     release(&p->lock);
501 }
```

sched

- `sched()` checks if process lock is acquired, no other lock is held, process is not running, and interrupt is disabled.

```
$ vim kernel/proc.c
```

```
493 void
494 yield(void)
495 {
496     struct proc *p = myproc();
497     acquire(&p->lock);
498     p->state = RUNNABLE;
499     sched();
500     release(&p->lock);
501 }
```

```
$ vim kernel/proc.c
```

```
472 void
473 sched(void)
474 {
475     int intena;
476     struct proc *p = myproc();
477
478     if(!holding(&p->lock))
479         panic("sched p->lock");
480     if(mycpu()->noff != 1)
481         panic("sched locks");
482     if(p->state == RUNNING)
483         panic("sched running");
484     if(intr_get())
485         panic("sched interruptible");
```

```
...
```

scheduler

- After `sched()` calls `swtch()`, stack pointer and the instruction address are returned (switched) to the CPU scheduler.
- Note that `sched` and `scheduler` have each of own kernel stack.

```
$ vim kernel/proc.c
```

```
472 void
473 sched(void)
474 {
475     int intena;
476     struct proc *p = myproc();
477     ...
487     intena = mycpu()->intena;
488     swtch(&p->context, &mycpu()->context);
489     mycpu()->intena = intena;
490 }
```

```
$ vim kernel/proc.c
```

```
424 void
425 scheduler(void)
426 {
427     ...
447     p->state = RUNNING;
448     c->proc = p;
449     swtch(&c->context, &p->context);
450     ...
463 }
```

context switch

Context switch in xv6

- In xv6, context switching is implemented through the `swtch()` function.
- `swtch()` takes two arguments.
 - `struct context *old`, `struct context *new`

```
$ vim kernel/swtch.S

8  .globl swtch
9  swtch:
10     sd ra, 0(a0)
11     sd sp, 8(a0)
12     sd s0, 16(a0)
13     sd s1, 24(a0)
...
23     sd s11, 104(a0)
24
25     ld ra, 0(a1)
26     ld sp, 8(a1)
27     ld s0, 16(a1)
...
38     ld s11, 104(a1)
39
40     ret
```

Context switch in xv6

- `switch()` goes through three steps:
 1. Save the current process's state(registers) in the `old` context.
 2. Load the register values from the `new` context.
 3. Execute the `ret` instruction, which follows the `ra`(return address), transferring control to the new process.

```
$ vim kernel/switch.S

8  .globl switch
9  switch:
10     sd ra, 0(a0)
11     sd sp, 8(a0)
12     sd s0, 16(a0)
13     sd s1, 24(a0)
...
23     sd s11, 104(a0)
24
25     ld ra, 0(a1)
26     ld sp, 8(a1)
27     ld s0, 16(a1)
...
38     ld s11, 104(a1)
39
40     ret
```

Scheduler in xv6

- In xv6, kernel thread always gives up its processor in `sched()`, and always switches to the same location in the scheduler.

```
$ vim kernel/proc.c
```

```
472 void
473 sched(void)
474 {
475     int intena;
476     struct proc *p = myproc();
477     ...
487     intena = mycpu()->intena;
488     swtch(&p->context, &mycpu()->context);
489     mycpu()->intena = intena;
490 }
```

context switch

```
$ vim kernel/proc.c
```

```
424 void
425 scheduler(void)
426 {
427     ...
431     for(;;){
428         ...
441         for(p = proc; p < &proc[NPROC]; p++){
442             acquire(&p->lock);
443             if(p->state == RUNNABLE) {
444                 ...
447                 p->state = RUNNING;
448                 c->proc = p;
449                 swtch(&c->context, &p->context);
450                 // comments
451                 c->proc = 0;
452                 found = 1;
453             }
454             release(&p->lock);
455         }
456     }
```

Scheduler in xv6

- In xv6, kernel thread always gives up its processor in `sched()`, and always switches to the same location in the scheduler.

```
$ vim kernel/proc.c
```

```
424 void
425 scheduler(void)
426 {
...

```

Note

- Lines 487, 488, and 489 in `sched` are not executed sequentially.
- As same, lines 448, 449, and 453 in `scheduler` are not executed sequentially.

```
$ vim kernel/proc.c
```

```
472 void
473 sched(void)
474 {
475     int intena;
476     struct proc *p = myproc();
...
487     intena = mycpu()->intena;
488     swtch(&p->context, &mycpu()->context);
489     mycpu()->intena = intena;
490 }
```

```
...
447         p->state = RUNNING;
448         c->proc = p;
449         swtch(&c->context, &p->context);
...
453         c->proc = 0;
454         found = 1;
455     }
456     release(&p->lock);

```

context switch

Scheduler in xv6

- In xv6, kernel thread always gives up its processor in `sched()`, and always switches to the same location in the scheduler.
- Scheduler in xv6 finds a runnable process, switches to it in for loop repeatedly.

```
$ vim kernel/proc.c

424 void
425 scheduler(void)
426 {
  ...
431   for(;;){
  ...
441       for(p = proc; p < &proc[NPROC]; p++){
442           acquire(&p->lock);
443           if(p->state == RUNNABLE) {
  ...
447               p->state = RUNNING;
448               c->proc = p;
449               swtch(&c->context, &p->context);
  ...
               // comments
453               c->proc = 0;
454               found = 1;
455           }
456       release(&p->lock);
```

Round-Robin Scheduling in xv6

- The xv6 scheduler uses a simple **Round-Robin** policy: on each tick, it switches to the next RUNNABLE process in the process array.
- After reaching the end of the array, the scheduler wraps around to the beginning.

```
$ vim kernel/proc.c

424 void
425 scheduler(void)
426 {
  ...
431   for(;;){
  ...
441       for(p = proc; p < &proc[NPROC]; p++){
442           acquire(&p->lock);
443           if(p->state == RUNNABLE) {
  ...
447               p->state = RUNNING;
448               c->proc = p;
449               swtch(&c->context, &p->context);
  ...
               // comments
453               c->proc = 0;
454               found = 1;
455           }
456           release(&p->lock);
```

Locks in scheduler

- In the scheduling process of xv6, `p->lock` is acquired by the process and is released later during the `yield` system call.
- After the context switch via `swtch()` in `sched()`, the `p->lock` acquired in `yield` is released in the scheduler.

```
$ vim kernel/proc.c
```

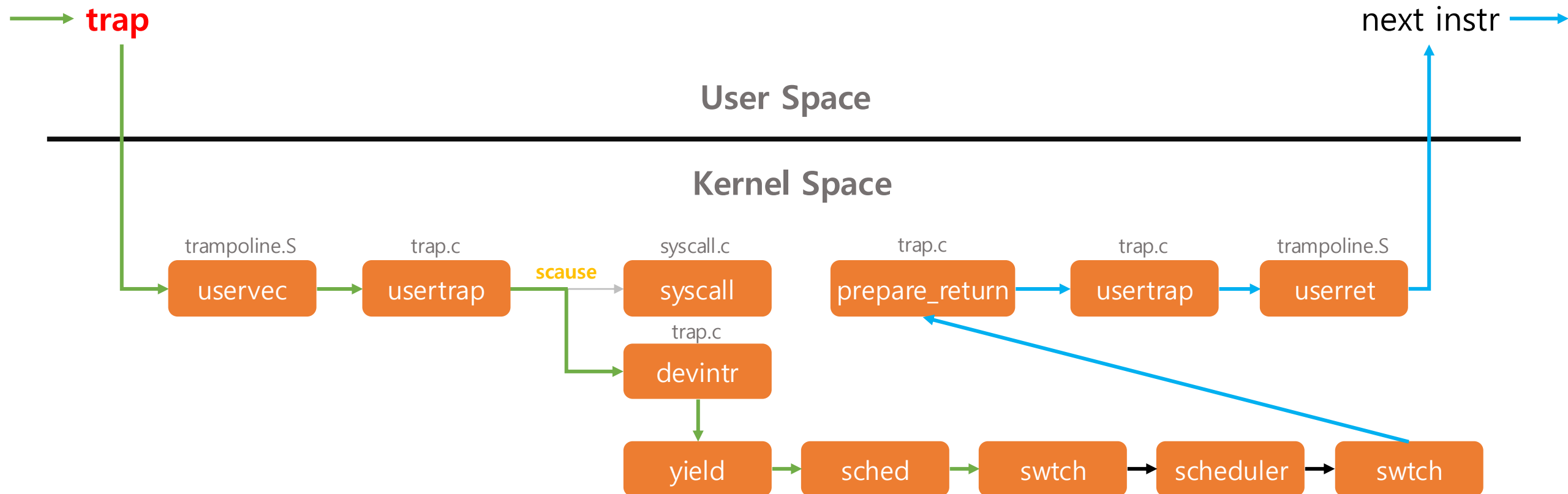
```
493 void
494 yield(void)
495 {
496     struct proc *p = myproc();
497     acquire(&p->lock);
498     ...
499     sched();
500     release(&p->lock);
501 }
```

context switch

```
$ vim kernel/proc.c
```

```
424 void
425 scheduler(void)
426 {
431     for(;;){
441         for(p = proc; p < &proc[NPROC]; p++)
442             acquire(&p->lock);
443             if(p->state == RUNNABLE) {
449                 swtch(&c->context, &p->context);
455             }
456             release(&p->lock);
457     }
```

Recap: Context Switch by Timer Interrupt



Practice on Your Own

scheduler debugging and voluntary yield

Branch Setup

- Make a branch named 'sandbox/lab05' in sandbox repository

```
$ cd <path_to_your_own_sandbox_repo>    # move to your sandbox repository
$ git switch sandbox/base                # switch to base branch
$ git branch sandbox/lab05               # make a new branch
$ git switch sandbox/lab05               # switch to lab05 branch
```

- TA's code will be uploaded in [sandbox/lab05](#) branch in the xv6-sandbox repository.

#1. Observing the Scheduler

- One basic way to debug is to simply print the variables, and check if they have intended values.
- Print current value of `ticks` variable, and the `pid` and the name of the process to the console, whenever the scheduler picks a process to run.
- Make a user program that runs an infinite loop and execute it.
- Run `xv6` with `CPUS=1`

```
$ make qemu CPUS=1
```

```
$ loop
ticks: 1000, pid: 3, name: loop
ticks: 1001, pid: 3, name: loop
ticks: 1002, pid: 3, name: loop
ticks: 1003, pid: 3, name: loop
...
```

Result Screenshot

```
make qemu CPUS=1
55% 19% 14 GB 7.2 kB↓ 10 kB↑
~/os/c/xv6-riscv-sandbox-for-TA sandbox/lab05 ?1 07:24:28 오후
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
ticks: 1, pid: 1, name:
init: starting sh
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: sh
$ticks: 1, pid: 2, name: sh
```

```
owunr@seongjunmoui-MacBookAir:~/os/c/xv6-riscv-sandbox-for-TA
55% 19% 14 GB 4.1 kB↓ 1.0 kB↑
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: init
ticks: 1, pid: 2, name: sh
$ticks: 1, pid: 2, name: sh
loop
ticks: 98, pid: 2, name: sh
ticks: 98, pid: 3, name: sh
ticks: 98, pid: 3, name: sh
ticks: 98, pid: 3, name: sh
ticks: 98, pid: 3, name: sh
ticks: 98, pid: 3, name: sh
ticks: 99, pid: 3, name: loop
ticks: 100, pid: 3, name: loop
ticks: 101, pid: 3, name: loop
ticks: 102, pid: 3, name: loop
ticks: 103, pid: 3, name: loop
ticks: 104, pid: 3, name: loop
ticks: 105, pid: 3, name: loop
ticks: 106, pid: 3, name: loop
ticks: 107, pid: 3, name: loop
ticks: 108, pid: 3, name: loop
ticks: 109, pid: 3, name: loop
ticks: 110, pid: 3, name: loop
ticks: 111, pid: 3, name: loop
ticks: 112, pid: 3, name: loop
ticks: 113, pid: 3, name: loop
ticks: 114, pid: 3, name: loop
ticks: 115, pid: 3, name: loop
ticks: 116, pid: 3, name: loop
ticks: 117, pid: 3, name: loop
ticks: 118, pid: 3, name: loop
ticks: 119, pid: 3, name: loop
ticks: 120, pid: 3, name: loop
ticks: 121, pid: 3, name: loop
ticks: 122, pid: 3, name: loop
ticks: 123, pid: 3, name: loop
QEMU: Terminated
~/os/c/xv6-riscv-sandbox-for-TA sandbox/lab05 ?1 13s 07:26:28 오후
```

Commit

- Changed Files

```
$ git status --short  
M Makefile  
M kernel/proc.c  
?? user/loop.c
```

- Add & Commit

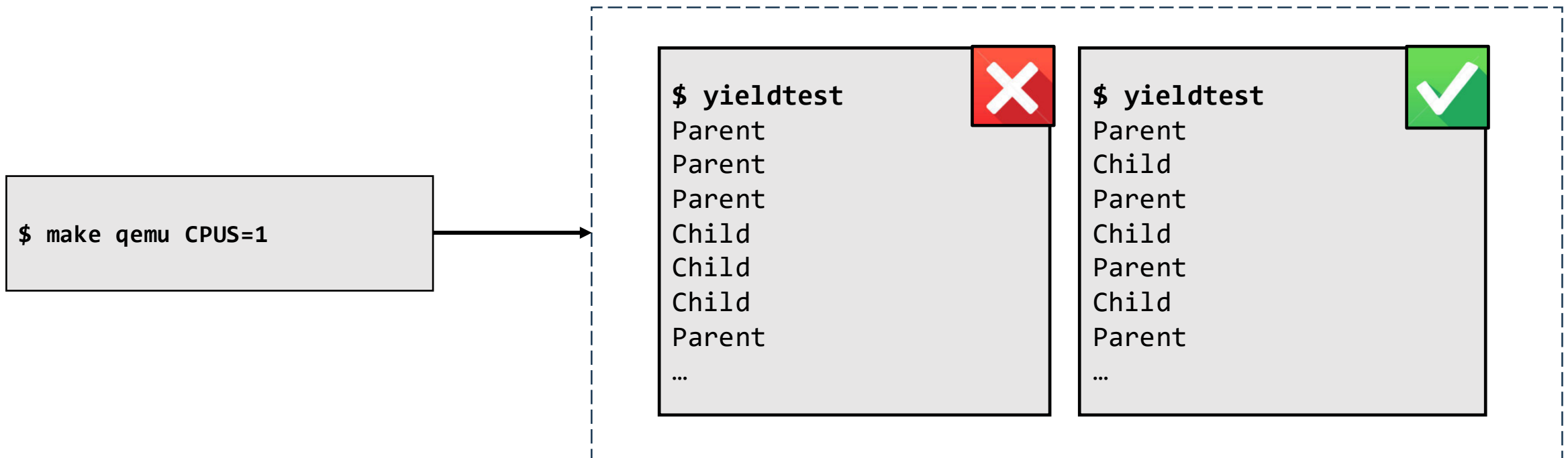
```
$ git add Makefile kernel/proc.c user/loop.c      # Stage all changes  
$ git commit -m "feat: scheduler debug print + loop program"  # Commit changes
```

#2. Voluntary CPU Yield

- In xv6, `yield` is implemented but not as a system call.
- First, implement `yield` system call so that user can call `yield`. (`yield` gives up its CPU)
- Make a user program in xv6 that creates child process with `fork()`.
 - Parent and child process print "Parent" and "Child" respectively in loop.

#2. Voluntary CPU Yield

- You can see that two running processes are switched periodically by timer interrupt.
- Make the result below using `yield` system call you implemented in the first step.
- Note that your user program should show the changed behavior.



Submission

- Changed Files

```
$ git status --short
```

```
M Makefile
M kernel/syscall.c
M kernel/syscall.h
M kernel/sysproc.c
M user/user.h
M user/usys.pl
?? user/yieldtest.c
```

- Commit & Push

```
$ git add . # Stage all changes
$ git commit -m "feat: add yield system call and test program" # Commit changes
$ git push origin sandbox/lab05 # Push to remote repository
```


Thank you

